

CSCI235 – Database Systems 2026 S2 Implementation Task 2 Due on 10 May 2026

Scope

The Implementation of Task 2 is related to the contents of topic on Indexing.

This Implementation is due by Sunday, 10 May 2026, 9:00 pm Singapore time. This task is worth 3% of the total assessment for the subject.

Only electronic submission through Moodle at: <https://moodle.uowplatform.edu.au/> is accepted. All email submission will be deleted and mark 0 ("zero") will be awarded.

For all the implemented tasks, your report or output must include a listing of all PL/pgSQL statements processed.

The submission procedure is explained at the end of this specification.

Objective

Two main objectives of this assignment are:

1. This assignment aims to help students understand how PostgreSQL uses indexes during query execution. You will learn how different types of index traversals occur (vertical vs horizontal) and how to distinguish between index scans and index-only scans based on query structure and column selection.
2. This assignment is designed to help students understand how PostgreSQL indexing improves query performance and enables index-only scans. Students will explore query plans, design efficient indexes, and analyse how PostgreSQL decides between index scan and index-only scan strategies.

Learning Objectives

- Understand the role and performance impact of indexes in PostgreSQL.
- Differentiate between sequential scan, index scan, and index-only scans.
- Identify suitable indexing strategies that minimize disk I/O and query execution time.
- Design indexes that make use of index-only scans for performance optimization.

Specification

If you have not done it yet, download the TPCHR sample database and load the sample TPCHR using user account **tpchr**. This will make the user account **tpchr** the owner of the TPCHR database.

Question 1 (1.0 mark)

Scenario

- a) Consider a relational database that consists of the relational tables created by the following CREATE TABLE statements:

```
CREATE TABLE PART(  
    P_PARTKEY      NUMBER(12)          NOT NULL,  
    P_NAME         VARCHAR(55)         NOT NULL,  
    P_RETAILPRICE  NUMBER(12,2)        NOT NULL,  
    CONSTRAINT PART_PKEY PRIMARY KEY (P_PARTKEY) );
```

```
CREATE TABLE SUPPLIER(  
    S_SUPPKEY      NUMBER(12)          NOT NULL,  
    S_NAME         CHAR(25)            NOT NULL,  
    S_NATIONKEY    NUMBER(12)          NOT NULL,  
    S_PHONE        CHAR(15)            NOT NULL,  
    CONSTRAINT SUPPLIER_PKEY PRIMARY KEY (S_SUPPKEY) );
```

```
CREATE TABLE PARTSUPP(  
    PS_PARTKEY     NUMBER(12)          NOT NULL,  
    PS_SUPPKEY     NUMBER(12)          NOT NULL,  
    PS_SUPPLYCOST  NUMBER(12,2)        NOT NULL,  
    CONSTRAINT PARTSUPP_PKEY PRIMARY KEY (PS_PARTKEY, PS_SUPPKEY),  
    CONSTRAINT PARTSUPP_FKEY1 FOREIGN KEY (PS_PARTKEY)  
        REFERENCES PART(P_PARTKEY),  
    CONSTRAINT PARTSUPP_FKEY2 FOREIGN KEY (PS_SUPPKEY)  
        REFERENCES SUPPLIER(S_SUPPKEY) );
```

For each query listed below, determine whether an index should be created to optimize its performance. Treat each query as an individual case.

- **If an index should be created:**
 - List the attributes that should form the index key.
 - Provide the CREATE INDEX statement to create the index.
 - Explain how the index will improve query performance. Note that the order of attributes in the index key is important.
- **If an index is not needed:**
 - Explain why query performance will not suffer without a new index.

Assume that all relational tables are large enough that full table scans are more time-consuming than indexed access.

- i. SELECT *
 FROM PART
 WHERE P_NAME = 'GRAPHIC CARD '
 AND P_RETAILPRICE = 250;

(0.2 mark)

The primary key of the relational table PART consists of the attribute P_PARTKEY. The system cannot use the primary key to speed up the information from the relational table PART. To speed up the query specified in part (1), we can create an index based on the attributes P_NAME and P_RETAILPRICE. The statement to create the index is as follow:

```
CREATE INDEX IDX1 ON PART(P_NAME, P_RETAILPRICE);
```

- ii. SELECT PS_SUPPKEY
 FROM PARTSUPP
 WHERE PS_PARTKEY = 'PMB001'

(0.2 mark)

The system can find the PS_PARTKEY in the primary key index, and since the attribute PS_PARTKEY is the first attribute of the composite primary key, the system is able to use the primary key of the PARTSUPP table to retrieve the required information. Hence, there is no need to create index.

- iii. SELECT S_NAME
 FROM SUPPLIER;

(0.2 mark)

The system cannot use the primary key index of the SUPPLIER table because the primary key index does not contain any information about the S_NAME. To speed up the query processing, we can create a secondary index as follow:

```
CREATE INDEX IDX3 ON SUPPLIER (S_NAME);
```

iv. `SELECT COUNT(*)
FROM PART
WHERE P_NAME = 'COOLING-FAN';`

(0.2 mark)

Vertical scan the index on `p_name = 'COOLING-FAN'`, then horizontal scan through leaf level of the index and count the number of rowid for all the `p_name = 'COOLING-FAN'`.

`CREATE INDEX IDX4 on PART(P_NAME);`

v. `SELECT PS_SUPPKEY, COUNT(*)
FROM PARTSUPP
GROUP BY PS_SUPPKEY
HAVING COUNT(PS_SUPPLYCOST) > 3;`

(0.2 mark)

Horizontal scan through a leaf level of the index and counting the distinct values of `PS_SUPPLYCOST` for each value of `PS_SUPPKEY`.

`CREATE INDEX partsuppIdx on PARTSUPP(PS_SUPPKEY, PS_SUPPLYCOST);`

Deliverables

Include all five SELECT statements in an SQL script file named IT2SolutionQ1.sql. For each query, include a report (IT2ReportQ1.pdf or IT2ReportQ1.txt or IT2ReportQ1.docx) explaining:

- Which part of the index is traversed (vertical / horizontal).
- Whether the query uses an index scan or index-only scan.
- Whether a **heap fetch (table access)** is required to satisfy the query, and provide the reasoning for your answer.

Question 2 (2.0 marks)

Using the relational table LINEITEM of the sample database TPCHR, for each one of the queries listed below:

- i. Find all the discount (I_discount) of all the items that are shipped (I_shipdate) most recently. Hint. Most recently mean the latest shipment date.
 - ii. Find the total number of items shipped by air (I_shipmode) in 1998 (I_shipdate).
 - iii. Find the order number (I_orderkey) and item number (I_linenum) that have the highest discount (I_discount).
 - iv. Find the total number of item per line status (I_linestatus). List the line status and the total items per line status.
 - v. Find the order key (I_orderkey), line item number (I_linenum), line status (I_linestatus), shipment date (I_shipdate) and shipment mode (I_shipmode) of all orders with the order number (I_orderkey) 1795718, 1799046, and 1794626.
- a) Construct an SQL statement that produces the required output specified in the statement. **(1.0 mark)**
- b) Find the **smallest** number of indexes that improve performance of a given collection of SELECT statements of a relational table LINEITEM. The smallest number of indexing means a database system will compute the five queries constructed in (a) using one or more indexes that you have created. Hint, you may create an index that can be used to compute more than one queries. Use the explain analyze and show plan statements to justify your solutions. **(1.0 mark)**

Please remember that **you must consider each one of the queries as an individual case!** Please remember that all relational tables are large enough to make full table scans more time consuming than accessing the tables through an index! It means that any solution in which an index is not used for query processing is incorrect.

Deliverables

Include all five SELECT statements in an SQL script file named IT2SolutionQ2.sql. For each query, include a report (IT2ReportQ2.pdf or IT2ReportQ2.txt or IT2ReportQ2.docx) containing the CREATE INDEX statements that improve the performance of the queries listed (i, ii, iii, iv, and v above) and the execution plan generated.

Sample solution includes but is not limited to the following:

```
/* =====
 * Implementation Task 2, Question 2
 * CSCI235, 2026S2
 * =====*/
-- Question 2.a.i
-- Find all the discount (l_discount) of all the items that
-- are shipped (l_shipdate) most recently. Hint. Most
-- recently mean the latest shipment date.
--
-- select l_discount
-- from lineitem
-- where l_shipdate in (
-- select max(l_shipdate)
-- from lineitem)
-- order by l_discount desc;
--
-- Question 2.a.ii
-- Find the total number of items shipped by air (l_shipmode)
-- in 1998 (l_shipdate).
--
-- select count(*)
-- from lineitem
-- where upper(l_shipmode) like '%AIR%'
-- and extract(year from l_shipdate) = '1998';
--
-- Question 2.a.iii
-- Find the order number (l_orderkey) and item number
-- (l_linenum) that have the highest discount (l_discount).
--
-- select l_orderkey, l_linenum
-- from lineitem
-- where l_discount = (
-- select max(l_discount)
-- from lineitem);
--
-- Question 2.a.iv
-- Find the total number of item per line status
-- (l_linestatus). List the line status and the total itmes
-- per line status.
--
-- select l_linestatus, count(*)
-- from lineitem
-- group by l_linestatus;
--
-- Question 2.a.v
-- Find the order key (l_orderkey), line item number
-- (l_linenum), line status (l_linestatus), shipment date
-- (l_shipmode) of all orders with the order number
```

```
-- (l_orderkey) 1795718, 1799046, and 1794626.
--
-- select l_orderkey, l_linenumber, l_linestatus, l_shipdate,
-- l_shipmode
-- from lineitem
-- where l_orderkey in (1795718, 1799046, 1794626);
--
--
-- Generate the execution plan for the query before the
-- creation of an index
--
-- Drop all the previously created indexes
DROP INDEX IF EXISTS IT2Q2Idx1;
DROP INDEX IF EXISTS IT2Q2Idx2;
DROP INDEX IF EXISTS IT2Q2Idx4;
DROP INDEX IF EXISTS IT2Q2Idx5;

-- Task 2.a.i
--
tpchr=> explain analyze
tpchr-> select l_discount, l_shipdate
tpchr-> from lineitem
tpchr-> where l_shipdate in (
tpchr(>      select max(l_shipdate) from lineitem);
                                QUERY PLAN
-----
Hash Join (cost=44140.72..100639.88 rows=718 width=8) (actual time=370.500..1041.456 rows=5.00
loops=1)
  Hash Cond: (lineitem.l_shipdate = (max(lineitem_1.l_shipdate)))
  Buffers: shared hit=29593 read=37937
  -> Seq Scan on lineitem (cost=0.00..51765.93 rows=1800093 width=8) (actual time=0.274..432.749
rows=1800093.00 loops=1)
    Buffers: shared hit=14936 read=18829
  -> Hash (cost=44140.71..44140.71 rows=1 width=4) (actual time=171.162..171.272 rows=1.00 loops=1)
    Buckets: 1024 Batches: 1 Memory Usage: 9kB
    Buffers: shared hit=14657 read=19108
  -> Finalize Aggregate (cost=44140.70..44140.71 rows=1 width=4) (actual time=171.152..171.262
rows=1.00 loops=1)
    Buffers: shared hit=14657 read=19108
  -> Gather (cost=44140.49..44140.69 rows=2 width=4) (actual time=171.009..171.255 rows=3.00
loops=1)
    Workers Planned: 2
    Workers Launched: 2
    Buffers: shared hit=14657 read=19108
  -> Partial Aggregate (cost=43140.49..43140.50 rows=1 width=4) (actual time=153.934..153.935
rows=1.00 loops=3)
    Buffers: shared hit=14657 read=19108
```

-> **Parallel Seq Scan on lineitem lineitem_1** (cost=0.00..41265.39 rows=750039 width=4)
(actual time=0.317..105.252 rows=600031.00 loops=3)
Buffers: shared hit=14657 read=19108

Planning:

Buffers: shared hit=16 dirtied=2

Planning Time: 0.674 ms

Execution Time: 1041.566 ms

(22 rows)

tpchr=>

-- **Task 2.a.ii**

--

tpchr=> **explain analyze**

tpchr-> **select count(l_shipmode)**

tpchr-> **from lineitem**

tpchr-> **where upper(l_shipmode) like '%AIR%'**

tpchr-> **and extract(year from l_shipdate) = '1998';**

QUERY PLAN

Finalize Aggregate (cost=51641.46..51641.47 rows=1 width=8) (actual time=216.037..220.508 rows=1.00 loops=1)

Buffers: shared hit=15030 read=18735

-> Gather (cost=51641.25..51641.46 rows=2 width=8) (actual time=215.834..220.501 rows=3.00 loops=1)

Workers Planned: 2

Workers Launched: 2

Buffers: shared hit=15030 read=18735

-> Partial Aggregate (cost=50641.25..50641.26 rows=1 width=8) (actual time=198.614..198.616 rows=1.00 loops=3)

Buffers: shared hit=15030 read=18735

-> **Parallel Seq Scan on lineitem** (cost=0.00..50640.87 rows=150 width=11) (actual time=0.892..197.527 rows=19625.67 loops=3)

Filter: ((EXTRACT(year FROM l_shipdate) = '1998'::numeric) AND (upper((l_shipmode)::text) ~ '%AIR%':text))

Rows Removed by Filter: 580405

Buffers: shared hit=15030 read=18735

Planning Time: 0.165 ms

Execution Time: 220.612 ms

(14 rows)

tpchr=>

-- **Task 2.a.iii**

--

tpchr=> **explain analyze**


```
tpchr-> select l_orderkey, l_linenumber
tpchr-> from lineitem
tpchr-> where l_discount = (
tpchr-> select max(l_discount)
tpchr-> from lineitem);
```

QUERY PLAN

```
-----
-----
-----
Seq Scan on lineitem (cost=44140.71..100406.87 rows=163645 width=8)
(actual time=173.952..895.530 rows=163868.00 loops=1)
  Filter: (l_discount = (InitPlan 1).col1)
  Rows Removed by Filter: 1636225
  Buffers: shared hit=30909 read=36621
  InitPlan 1
    -> Finalize Aggregate (cost=44140.70..44140.71 rows=1 width=32)
      (actual time=173.653..174.048 rows=1.00 loops=1)
        Buffers: shared hit=15315 read=18450
        -> Gather (cost=44140.49..44140.69 rows=2 width=32) (actual
time=173.483..174.040 rows=3.00 loops=1)
          Workers Planned: 2
          Workers Launched: 2
          Buffers: shared hit=15315 read=18450
          -> Partial Aggregate (cost=43140.49..43140.50 rows=1
width=32) (actual time=154.551..154.551 rows=1.00 loops=3)
            Buffers: shared hit=15315 read=18450
            -> Parallel Seq Scan on lineitem lineitem_1
              (cost=0.00..41265.39 rows=750039 width=4) (actual time=0.420..95.673
rows=600031.00 loops=3)
                Buffers: shared hit=15315 read=18450

Planning Time: 0.340 ms
Execution Time: 909.779 ms
(17 rows)
```

```
tpchr=>
-- Task 2.a.iv
--
tpchr=> explain analyze
tpchr-> select l_linestatus, count(l_linestatus)
tpchr-> from lineitem
tpchr-> group by l_linestatus;
```

QUERY PLAN

```
-----
-----
---
Finalize GroupAggregate (cost=46015.64..46016.26 rows=2 width=10)
(actual time=186.229..190.644 rows=2.00 loops=1)
  Group Key: l_linestatus
```

```

Buffers: shared hit=15702 read=18077
-> Gather Merge (cost=46015.64..46016.22 rows=5 width=10) (actual
time=186.216..190.634 rows=6.00 loops=1)
  Workers Planned: 2
  Workers Launched: 2
  Buffers: shared hit=15702 read=18077
  -> Sort (cost=45015.61..45015.62 rows=2 width=10) (actual
time=167.140..167.141 rows=2.00 loops=3)
    Sort Key: l_linestatus
    Sort Method: quicksort Memory: 25kB
    Buffers: shared hit=15702 read=18077
    Worker 0: Sort Method: quicksort Memory: 25kB
    Worker 1: Sort Method: quicksort Memory: 25kB
    -> Partial HashAggregate (cost=45015.58..45015.60 rows=2
width=10) (actual time=167.072..167.073 rows=2.00 loops=3)
      Group Key: l_linestatus
      Batches: 1 Memory Usage: 32kB
      Buffers: shared hit=15688 read=18077
      Worker 0: Batches: 1 Memory Usage: 32kB
      Worker 1: Batches: 1 Memory Usage: 32kB
      -> Parallel Seq Scan on lineitem
(cost=0.00..41265.39 rows=750039 width=2) (actual time=0.411..76.991
rows=600031.00 loops=3)
        Buffers: shared hit=15688 read=18077

Planning Time: 0.147 ms
Execution Time: 190.768 ms
(23 rows)

```

```

tpchr=>
-- Task 2.a.v
--
tpchr=> explain analyze
tpchr-> select l_orderkey, l_linenumber, l_linestatus, l_shipdate,
tpchr-> l_shipmode
tpchr-> from lineitem
tpchr-> where l_orderkey in (1795718, 1799046, 1794626);

```

QUERY PLAN

```

-----
Index Scan using lineitem_pkey on lineitem (cost=0.43..43.81 rows=16
width=25) (actual time=0.135..0.168 rows=14.00 loops=1)
  Index Cond: (l_orderkey = ANY
('{'1795718,1799046,1794626}'::integer[]))
  Index Searches: 3
  Buffers: shared hit=7 read=5
  Planning Time: 0.105 ms
Execution Time: 0.184 ms
(6 rows)

```

tpchr=>

-- Create the necessary indexes

tpchr=> create index IT2Q2Idx1 on lineitem(l_shipdate, l_discount);

CREATE INDEX

tpchr=> create index IT2Q2Idx2 on lineitem(l_shipmode, extract(year from l_shipdate));

CREATE INDEX

tpchr=> create index IT2Q2Idx4 on lineitem(l_linestatus);

CREATE INDEX

tpchr=> create index IT2Q2Idx5 on LINEITEM(l_orderkey, l_linenum, l_discount, l_shipdate, l_linestatus, l_shipmode);

CREATE INDEX

tpchr=>

--

-- Task 2.a.i

--

tpchr=> explain analyze

tpchr-> select l_discount, l_shipdate

tpchr-> from lineitem

tpchr-> where l_shipdate in (

tpchr-> select max(l_shipdate) from lineitem);

QUERY PLAN

```

-----
-----
-----
Nested Loop  (cost=0.89..32.64 rows=718 width=8) (actual
time=0.100..0.101 rows=5.00 loops=1)
  Buffers: shared hit=5 read=3
  -> Result  (cost=0.46..0.47 rows=1 width=4) (actual time=0.094..0.095
rows=1.00 loops=1)
    Buffers: shared hit=1 read=3
    InitPlan 1
      -> Limit  (cost=0.43..0.46 rows=1 width=4) (actual
time=0.092..0.092 rows=1.00 loops=1)
        Buffers: shared hit=1 read=3
        -> Index Only Scan Backward using it2q2idx1 on lineitem
lineitem_1  (cost=0.43..54045.82 rows=1800093 width=4) (actual
time=0.091..0.091 rows=1.00 loops=1)
          Heap Fetches: 0
          Index Searches: 1
          Buffers: shared hit=1 read=3
        -> Index Only Scan using it2q2idx1 on lineitem (cost=0.43..24.99
rows=718 width=8) (actual time=0.005..0.005 rows=5.00 loops=1)
          Index Cond: (l_shipdate = ((InitPlan 1).col1))
          Heap Fetches: 0
          Index Searches: 1
          Buffers: shared hit=4

```

```
Planning:
  Buffers: shared hit=80 read=4 dirtied=2
Planning Time: 2.198 ms
Execution Time: 0.128 ms
(20 rows)
```

```
tpchr=>
-- Task 2.a.ii
--
tpchr=> explain analyze
tpchr-> select count(l_shipmode)
tpchr-> from lineitem
tpchr-> where upper(l_shipmode) like '%AIR%'
tpchr-> and extract(year from l_shipdate) = '1998';
```

QUERY PLAN

```
-----
Aggregate (cost=20647.07..20647.08 rows=1 width=8) (actual
time=445.489..445.491 rows=1.00 loops=1)
  Buffers: shared hit=118 read=29328
  -> Bitmap Heap Scan on lineitem (cost=231.01..20646.17 rows=360
width=11) (actual time=17.372..436.851 rows=58877.00 loops=1)
    Recheck Cond: (EXTRACT(year FROM l_shipdate) = '1998'::numeric)
    Filter: (upper((l_shipmode)::text) ~~ '%AIR%'::text)
    Rows Removed by Filter: 148196
    Heap Blocks: exact=28622
    Buffers: shared hit=118 read=29328
    -> Bitmap Index Scan on it2q2idx2 (cost=0.00..230.92 rows=9000
width=0) (actual time=14.723..14.724 rows=207073.00 loops=1)
      Index Cond: (EXTRACT(year FROM l_shipdate) =
'1998'::numeric)
      Index Searches: 8
      Buffers: shared hit=7 read=817
```

```
Planning:
  Buffers: shared read=2
Planning Time: 0.307 ms
Execution Time: 445.966 ms
(16 rows)
```

```
tpchr=>
-- Task 2.a.iii
--
tpchr=> explain analyze
tpchr-> select l_orderkey, l_linenumber
tpchr-> from lineitem
tpchr-> where l_discount = (
tpchr-> select max(l_discount)
tpchr-> from lineitem);
```

QUERY PLAN

```

-----
-----
-----
Bitmap Heap Scan on lineitem (cost=54107.27..89917.83 rows=163645
width=8) (actual time=325.303..606.108 rows=163868.00 loops=1)
  Recheck Cond: (l_discount = (InitPlan 1).col1)
  Heap Blocks: exact=33567
  Buffers: shared hit=20836 read=55095 written=2
  InitPlan 1
    -> Finalize Aggregate (cost=44140.70..44140.71 rows=1 width=32)
(actual time=128.579..128.737 rows=1.00 loops=1)
  Buffers: shared hit=15988 read=17777
    -> Gather (cost=44140.49..44140.69 rows=2 width=32) (actual
time=128.570..128.730 rows=3.00 loops=1)
      Workers Planned: 2
      Workers Launched: 2
      Buffers: shared hit=15988 read=17777
    -> Partial Aggregate (cost=43140.49..43140.50 rows=1
width=32) (actual time=111.401..111.402 rows=1.00 loops=3)
      Buffers: shared hit=15988 read=17777
    -> Parallel Seq Scan on lineitem lineitem_1
(cost=0.00..41265.39 rows=750039 width=4) (actual time=0.311..54.894
rows=600031.00 loops=3)
      Buffers: shared hit=15988 read=17777
    -> Bitmap Index Scan on it2q2idx1 (cost=0.00..9925.65 rows=163645
width=0) (actual time=314.667..314.668 rows=163868.00 loops=1)
      Index Cond: (l_discount = (InitPlan 1).col1)
      Index Searches: 2445
      Buffers: shared hit=20836 read=21528
Planning Time: 0.171 ms
Execution Time: 620.574 ms
(21 rows)

```

```

tpchr=>
-- Task 2.a.iv
--
tpchr=> explain analyze
tpchr-> select l_linestatus, count(l_linestatus)
tpchr-> from lineitem
tpchr-> group by l_linestatus;

```

QUERY PLAN

```

-----
-----
-----
Finalize GroupAggregate (cost=1000.45..27356.14 rows=2 width=10)
(actual time=88.394..91.382 rows=2.00 loops=1)
  Group Key: l_linestatus

```

```

Buffers: shared hit=9 read=1520
-> Gather Merge (cost=1000.45..27356.10 rows=5 width=10) (actual
time=88.020..91.370 rows=6.00 loops=1)
    Workers Planned: 2
    Workers Launched: 2
    Buffers: shared hit=9 read=1520
    -> Partial GroupAggregate (cost=0.43..26355.50 rows=2
width=10) (actual time=31.533..56.211 rows=2.00 loops=3)
        Group Key: l_linestatus
        Buffers: shared hit=9 read=1520
        -> Parallel Index Only Scan using it2q2idx4 on lineitem
(cost=0.43..22605.28 rows=750039 width=2) (actual time=0.250..28.093
rows=600031.00 loops=3)
            Heap Fetches: 0
            Index Searches: 1
            Buffers: shared hit=9 read=1520
Planning Time: 0.102 ms
Execution Time: 91.420 ms
(16 rows)

```

```

tpchr=>
-- Task 2.a.v
--
tpchr=> explain analyze
tpchr-> select l_orderkey, l_linenumber, l_linestatus, l_shipdate,
tpchr-> l_shipmode
tpchr-> from lineitem
tpchr-> where l_orderkey in (1795718, 1799046, 1794626);

```

QUERY PLAN

```

-----
Index Only Scan using it2q2idx5 on lineitem (cost=0.43..13.56 rows=16
width=25) (actual time=0.124..0.172 rows=14.00 loops=1)
    Index Cond: (l_orderkey = ANY
({'1795718,1799046,1794626'}::integer[]))
    Heap Fetches: 0
    Index Searches: 3
    Buffers: shared hit=5 read=5
    Planning Time: 0.140 ms
    Execution Time: 0.191 ms
(7 rows)

```

```
tpchr=>
```

Submissions

This assignment is due by 9:00 pm (21:00 hours) **Sunday, 10 May 2026 at 9:00 pm Singapore time.**

Zip the files IT2SolutionQ1.sql, IT2SolutionQ2.sql, IT2ReportQ1.pdf, and IT2ReportQ2.pdf into IT2Solution.zip and submit the zipped file through Moodle in the following way:

- 1) Access Moodle at **<http://moodle.uowplatform.edu.au/>**
- 2) To login use a Login link located in the right upper corner the Web page or in the middle of the bottom of the Web page
- 3) When successfully logged in, select a site CSCI235 (SP226) Database Systems
- 4) Scroll down to a section Submissions of Implementation Tasks
- 5) Click at Submit your Implementation Task 1 here link.
- 6) Click at a button Add Submission
- 7) Move the .sql and .pdf (or .docx) or IT2Solution.zip (**or the zipped file**) into an area provided in Moodle. You can drag and drop files here to add them. You can also use a link *Add...*
- 8) Click at a button Save changes,
- 9) Click at check box to confirm authorship of a submission,
- 10) When you are satisfied, remember to click at a button Submit assignment.

A policy regarding late submissions is included in the subject outline.

Only one submission per student is accepted.

Implementation Task 2 is an individual assessed task and it is expected that all its tasks will be solved individually without any cooperation with the other students. Plagiarism is treated seriously. Students involved will likely receive zero. If you have any doubts, questions, etc. please consult your lecturer or tutor during lab classes or over e-mail.

End of specification